

assingment_key_hardcore

Your Name

Load Required Packages

```
library(tidyverse)    # For data manipulation and visualization
```

```
— Attaching core tidyverse packages ————— tidyverse 2.0.0 —
✓ dplyr      1.1.4      ✓ readr      2.1.5
✓ forcats    1.0.0      ✓ stringr    1.5.1
✓ ggplot2    3.5.2      ✓ tibble     3.3.0
✓ lubridate  1.9.4      ✓ tidyr      1.3.1
✓ purrr      1.1.0
— Conflicts ————— tidyverse_conflicts() —
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
! Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(car)          # For regression diagnostics (VIF, etc.)
```

Loading required package: carData

Attaching package: 'car'

The following object is masked from 'package:dplyr':

recode

The following object is masked from 'package:purrr':

some

```
library(corrplot)     # For correlation plots
```

corrplot 0.95 loaded

```
library(GGally)       # For pairs plots
library(broom)         # For tidy model outputs
library(performance)  # For model performance metrics
library(see)           # For better diagnostic plots
library(janitor)       # For clean names
```

Attaching package: 'janitor'

The following objects are masked from 'package:stats':

chisq.test, fisher.test

```
library(skimr)          # For data summary
```

Load and Prepare Data

```
# Load the dataset
lake_df <- read_csv("data/cheruvelil_lake_data.csv") %>%
  clean_names()
```

Rows: 593 Columns: 16

— Column specification —

Delimiter: ",",

chr (7): lake, state, source, state_lake_id, epa_lake_id, date, AgUrb.prop

dbl (9): longitude_utm, latitude_utm, year, CHL_uGL, TN_uGL, TP_uGL, Area_km...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Convert categorical variable to numeric
```

```
lake_df <- lake_df %>%
  mutate(
    watershed_proportion = case_when(
      ag_urb_prop == "low" ~ 0,
      ag_urb_prop == "high" ~ 1,
      TRUE ~ NA_real_
    )
  )
```

```
# Display basic information about the dataset
```

```
glimpse(lake_df)
```

Rows: 593

Columns: 17

\$ lake	<chr> "Lacey Keosauqua Park Lake", "Lake Geode", "Alcon...
\$ longitude_utm	<dbl> 337515.4, 385681.0, 962961.1, 801077.7, 774319.8,...
\$ latitude_utm	<dbl> 1973360, 1987587, 2459749, 2457101, 2437134, 2451...
\$ state	<chr> "IA", "IA", "MI", "MI", "MI", "MI", "MI", "MI", "...
\$ source	<chr> "IADNR_ISU", "IADNR_ISU", "MI_STORET", "MI_DNR_ST...
\$ state_lake_id	<chr> "50", "55", "1_4", "10_111", "10_24", "10_39", "1...
\$ epa_lake_id	<chr> "IA050", "IA055", "MI1_4", "MI10_111", "MI10_24",...
\$ date	<chr> "8/4/04", "8/3/04", "8/25/80", NA, NA, "8/12/82",...
\$ year	<dbl> 2004, 2004, 1980, 2004, 2004, 1982, 2004, 1980, 1...
\$ chl_uGL	<dbl> 11.15, 10.63, 2.10, 1.00, 4.20, 1.70, 1.00, 0.62,...
\$ tn_uGL	<dbl> 2714, 1651, 219, 252, 377, 256, 178, 822, 309, 21...
\$ tp_uGL	<dbl> 22.14, 20.79, 4.00, 4.00, 10.00, 7.50, 9.00, 19.0...
\$ area_km2	<dbl> 8.350, 71.281, 392.189, 202.485, 180.513, 1024.43...
\$ z_mean_m	<dbl> 3.561, 7.216, 5.000, 9.700, 8.900, 8.230, 17.500,...
\$ watershed_km2	<dbl> 2.984, 41.021, 3990.403, 110.500, 58.984, 462.643...

```
$ ag_urb_prop      <chr> "low", "low", "low", "low", "low", "low", "low", ...
$ watershed_proportion <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
```

Exploratory Data Analysis

```
# Create summary statistics
lake_df %>%
  select(chl_ug_l, tn_ug_l, tp_ug_l, area_km2, z_mean_m,
         watershed_km2, watershed_proportion) %>%
  skim()
```

Name Piped data

Number of rows 593

Number of columns 7

Column type frequency:

numeric 7

Group variables None

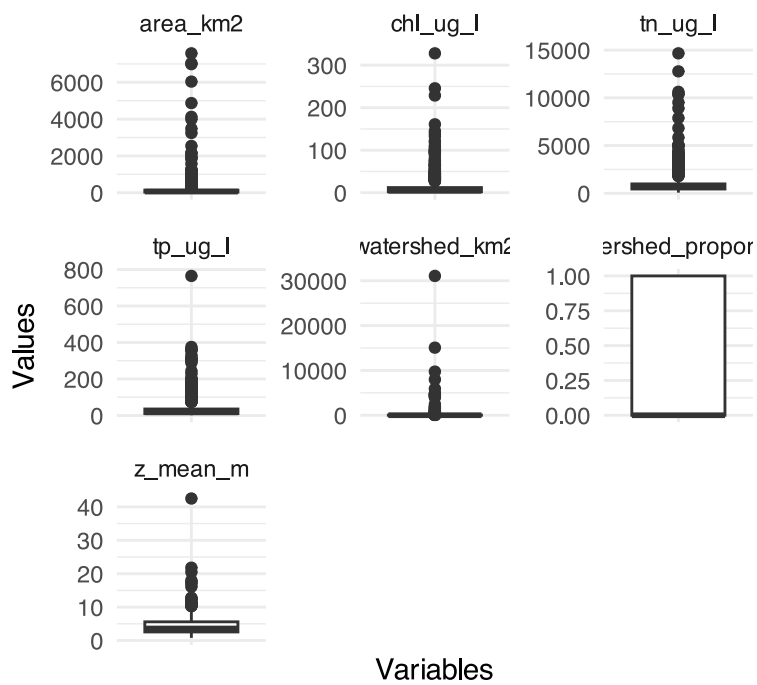
Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
chl_ug_l	0	1	14.89	29.35	0.10	2.20	4.50	12.10	327.84	
tn_ug_l	0	1	1052.63	1441.12	66.00	447.00	648.00	984.00	14661.00	
tp_ug_l	0	1	38.18	62.71	1.00	10.00	17.00	35.32	765.00	
area_km2	0	1	226.92	710.38	4.18	34.91	64.56	142.79	7575.07	
z_mean_m	0	1	4.46	3.16	0.80	2.57	3.60	5.60	42.50	
watershed_km2	0	1	244.74	1614.71	0.02	2.89	9.12	39.38	31079.86	
watershed_proportion	0	1	0.34	0.47	0.00	0.00	0.00	1.00	1.00	

```
# Create boxplots to examine distributions
lake_long <- lake_df %>%
  select(chl_ug_l, tn_ug_l, tp_ug_l, area_km2, z_mean_m,
         watershed_km2, watershed_proportion) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value")

ggplot(lake_long, aes(x = variable, y = value)) +
  geom_boxplot() +
  facet_wrap(~variable, scales = "free") +
  theme_minimal() +
  theme(axis.text.x = element_blank()) +
  labs(title = "Distribution of Original Variables",
       x = "Variables", y = "Values")
```

Distribution of Original Variables

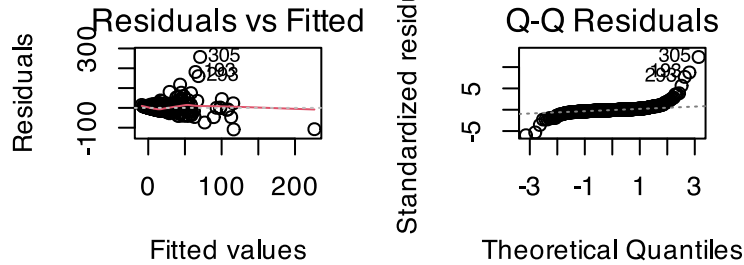


Step 1: Initial Model with Untransformed Variables

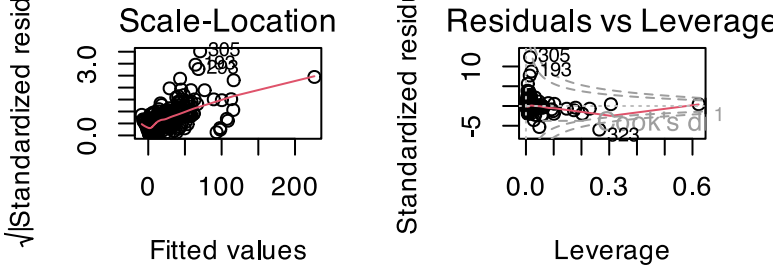
```
# Fit initial model with all untransformed variables
model_initial <- lm(chl_ug_l ~ tn_ug_l + tp_ug_l + area_km2 +
                    z_mean_m + watershed_km2 + watershed_proportion,
                    data = lake_df)

# Check initial model diagnostics
par(mfrow = c(2, 2))
plot(model_initial, main = "Initial Model Diagnostics (Untransformed)")
```

Model Diagnostics (Untransformed)



Model Diagnostics (Untransformed)



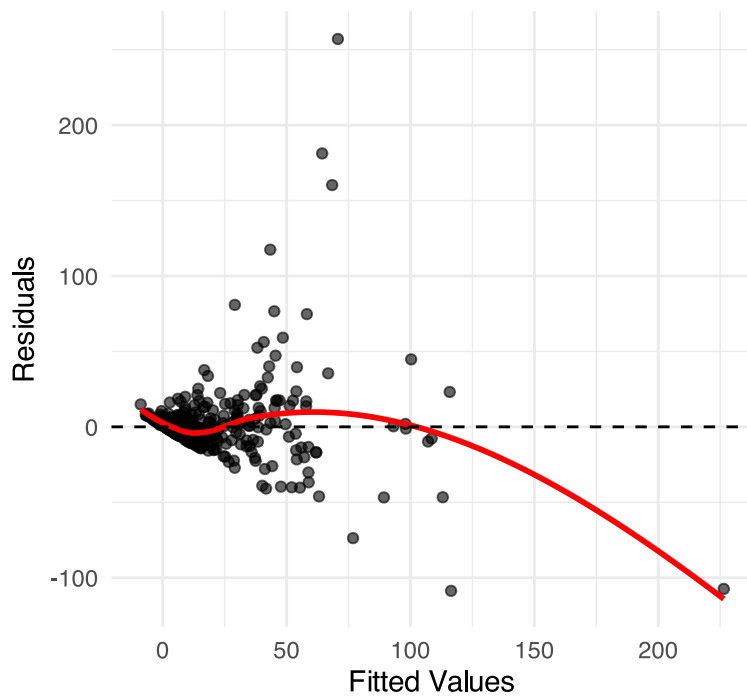
```
par(mfrow = c(1, 1))
```

```
# Examine residuals more closely
model_data_initial <- augment(model_initial)

# Residuals vs Fitted - check for heteroscedasticity
ggplot(model_data_initial, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(title = "Residuals vs Fitted (Untransformed)",
       x = "Fitted Values", y = "Residuals") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```

Residuals vs Fitted (Untransformed)



Step 2: Transform Only Response Variable

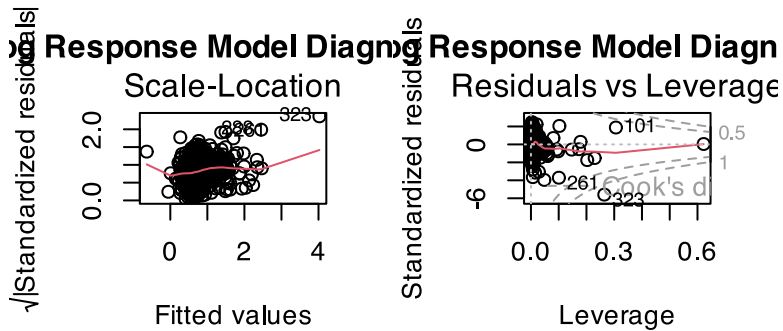
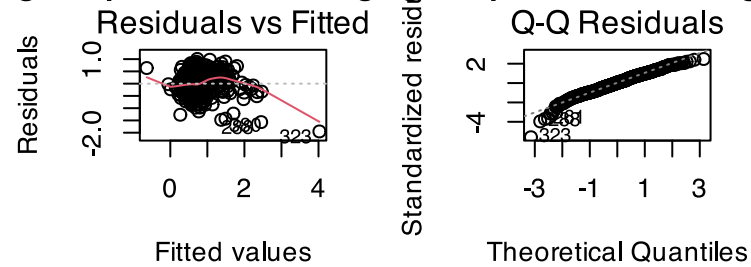
The diagnostic plots show heteroscedasticity (fan-shaped residuals), suggesting we need to transform the response variable. Let's try log-transforming only the response variable first.

```
# Transform only the response variable
lake_df <- lake_df %>%
  mutate(log_chl_ug_l = log10(chl_ug_l))

# Fit model with log-transformed response, untransformed predictors
model_log_response <- lm(log_chl_ug_l ~ tn_ug_l + tp_ug_l + area_km2 +
  z_mean_m + watershed_km2 + watershed_proportion,
  data = lake_df)

# Check diagnostics
par(mfrow = c(2, 2))
plot(model_log_response, main = "Log Response Model Diagnostics")
```

Log Response Model Diagnostics



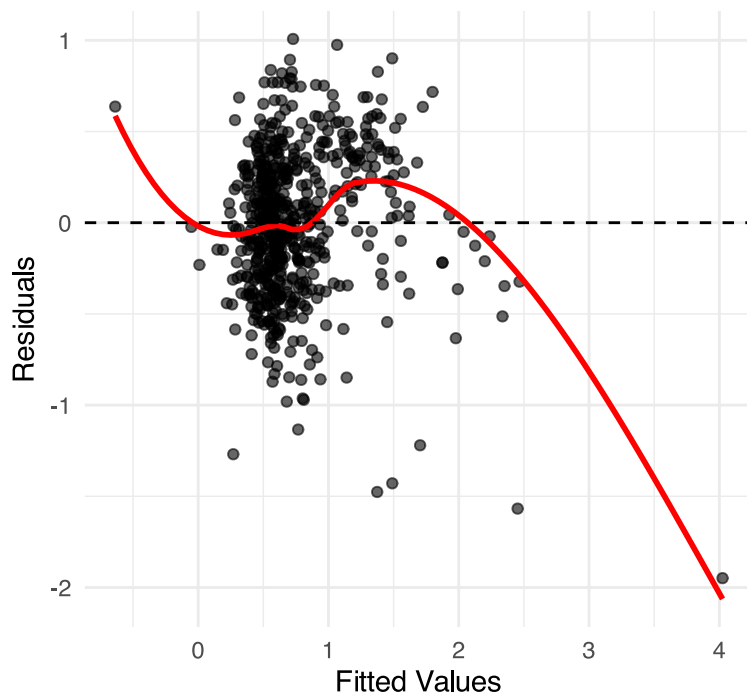
```
par(mfrow = c(1, 1))
```

```
# Examine residuals for log-response model
model_data_log <- augment(model_log_response)

# Residuals vs Fitted
ggplot(model_data_log, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(title = "Residuals vs Fitted (Log Response Only)",
       x = "Fitted Values", y = "Residuals") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```

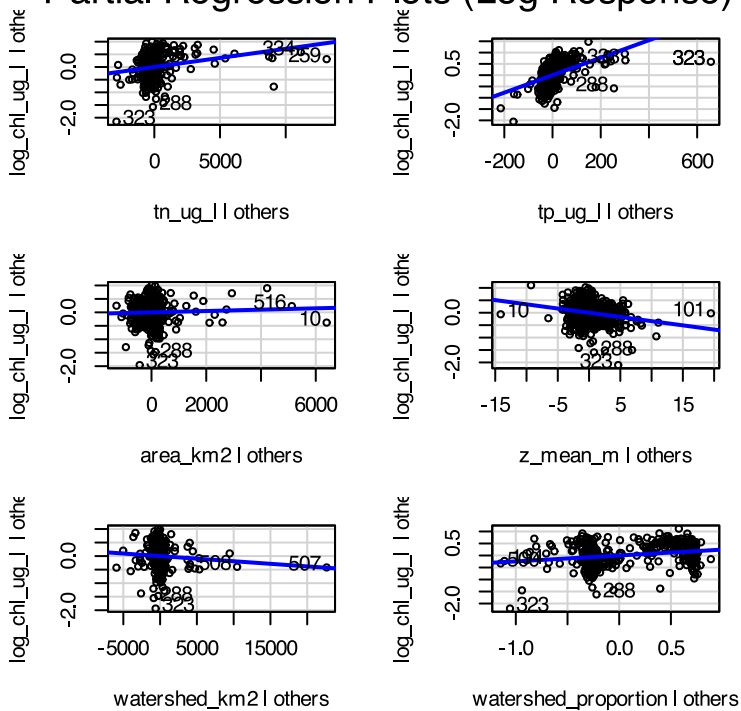
Residuals vs Fitted (Log Response Only)



Step 3: Check if Predictor Transformations are Needed

```
# Create partial regression plots to check for non-linearity
par(mfrow = c(2, 3))
avPlots(model_log_response, main = "Partial Regression Plots (Log Response)")
```

Partial Regression Plots (Log Response)



```
par(mfrow = c(1, 1))
```

Systematic Transformation Testing BEFORE Multiple Regression

Step 1: Start with Response Variable Transformation

```
# First, determine if response needs transformation
model_original_response <- lm(chl_ug_l ~ tn_ug_l + tp_ug_l + area_km2 +
                             z_mean_m + watershed_km2 + watershed_proportion,
                             data = lake_df)

# Add log-transformed response
lake_df <- lake_df %>%
  mutate(log_chl_ug_l = log10(chl_ug_l))

model_log_response <- lm(log_chl_ug_l ~ tn_ug_l + tp_ug_l + area_km2 +
                         z_mean_m + watershed_km2 + watershed_proportion,
                         data = lake_df)

# Quick diagnostic comparison
cat("=== Response Transformation Decision ===\n")
```

```
=== Response Transformation Decision ===
```

```
cat("Original response AIC:", round(AIC(model_original_response), 2), "\n")
```

```
Original response AIC: 5299.87
```

```
cat("Log response AIC:", round(AIC(model_log_response), 2), "\n")
```

```
Log response AIC: 623.49
```

```
cat("AIC improvement:", round(AIC(model_original_response) - AIC(model_log_response), 2), "\n")
```

```
AIC improvement: 4676.38
```

```
# Use log response as baseline (typical for ecological data)
baseline_model <- model_log_response
```

Step 2: Test Each Predictor Transformation Individually

```
# Function to test individual predictor transformations
test_predictor_transformation <- function(predictor_name, baseline_model, data) {

  # Create log-transformed version of predictor
  log_var_name <- paste0("log_", predictor_name)
  data[[log_var_name]] <- log10(data[[predictor_name]])

  # Get baseline formula and create transformed version
  baseline_formula <- formula(baseline_model)

  # Create new formula with log-transformed predictor
```

```

new_formula <- update(baseline_formula,
                      as.formula(paste("~ . -", predictor_name, "+", log_var_name)))

# Fit transformed model
transformed_model <- lm(new_formula, data = data)

# Compare AIC
baseline_aic <- AIC(baseline_model)
transformed_aic <- AIC(transformed_model)
delta_aic <- baseline_aic - transformed_aic

# Return results
return(list(
  predictor = predictor_name,
  baseline_aic = round(baseline_aic, 2),
  transformed_aic = round(transformed_aic, 2),
  delta_aic = round(delta_aic, 2),
  improvement = delta_aic > 2,
  model = transformed_model
))
}

# Test each predictor
predictors_to_test <- c("tn_ug_l", "tp_ug_l", "area_km2", "z_mean_m", "watershed_km2")

transformation_results <- list()
for(pred in predictors_to_test) {
  transformation_results[[pred]] <- test_predictor_transformation(pred, baseline_model, lake_df)
}

# Create summary table
transformation_summary <- data.frame(
  Predictor = sapply(transformation_results, function(x) x$predictor),
  Baseline_AIC = sapply(transformation_results, function(x) x$baseline_aic),
  Log_Transform_AIC = sapply(transformation_results, function(x) x$transformed_aic),
  Delta_AIC = sapply(transformation_results, function(x) x$delta_aic),
  Worthwhile = sapply(transformation_results, function(x) x$improvement),
  Decision = sapply(transformation_results, function(x) {
    if(x$delta_aic > 2) "✓ Transform"
    else if(x$delta_aic > 0) "? Maybe"
    else "✗ Don't transform"
  })
)

print("=== Individual Predictor Transformation Results ===")

```

```
[1] "=== Individual Predictor Transformation Results ==="
```

```
transformation_summary[order(-transformation_summary$Delta_AIC), ]
```

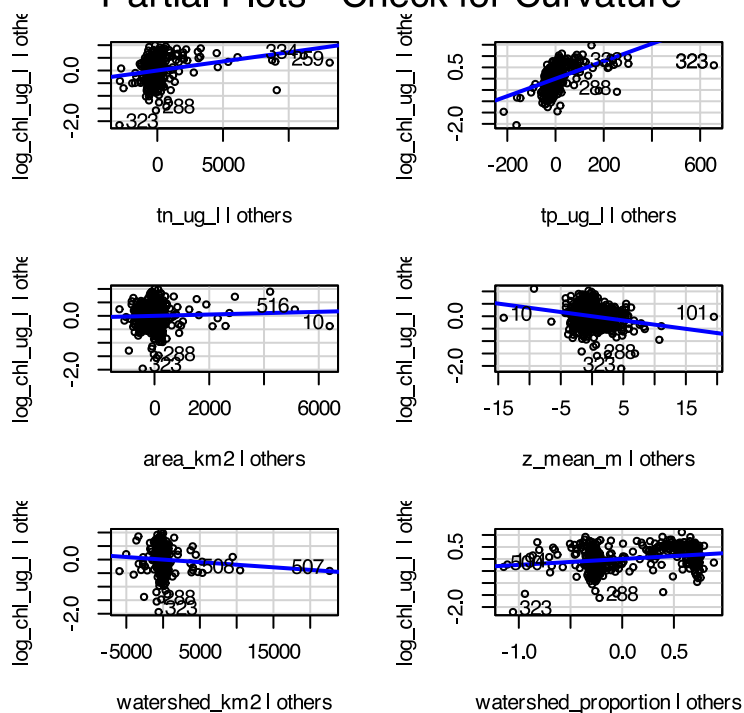
	Predictor	Baseline_AIC	Log_Transform_AIC	Delta_AIC	Worthwhile
tp_ug_l	tp_ug_l	623.49	492.35	131.14	TRUE
tn_ug_l	tn_ug_l	623.49	570.37	53.12	TRUE
area_km2	area_km2	623.49	620.98	2.51	TRUE

watershed_km2	watershed_km2	623.49	625.43	-1.94	FALSE
z_mean_m	z_mean_m	623.49	634.21	-10.72	FALSE
Decision					
tp_ug_l		✓ Transform			
tn_ug_l		✓ Transform			
area_km2		✓ Transform			
watershed_km2	x	Don't transform			
z_mean_m	x	Don't transform			

Step 3: Visual Check with Partial Plots

```
# Create partial plots for baseline model to check for curvature
par(mfrow = c(2, 3))
avPlots(baseline_model, main = "Partial Plots - Check for Curvature")
```

Partial Plots - Check for Curvature



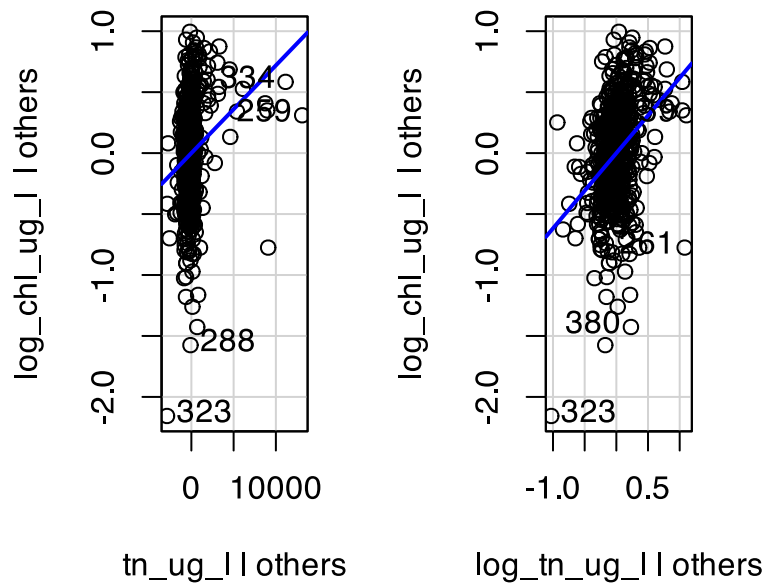
```
par(mfrow = c(1, 1))

# Function to create before/after partial plots for transformed predictors
check_transformation_visually <- function(predictor_name, results) {
  if(results$improvement) {
    cat("\n=== Visual check for", predictor_name, "transformation ===\n")

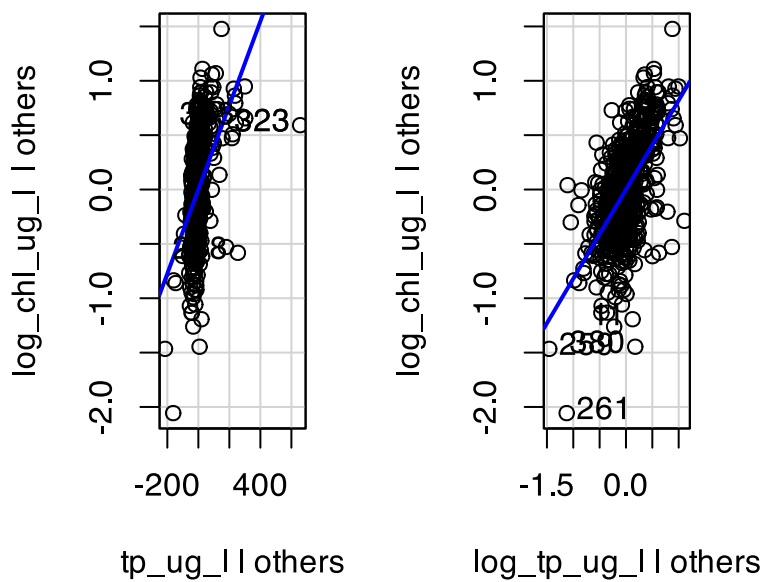
    # Before transformation
    par(mfrow = c(1, 2))
    avPlots(baseline_model, terms = predictor_name, main = paste("Before:", predictor_name))
    avPlots(results$model, terms = paste0("log_", predictor_name), main = paste("After: log",
predictor_name))
    par(mfrow = c(1, 1))
  }
}
```

```
# Check visual improvements for worthwhile transformations
for(pred in names(transformation_results)) {
  check_transformation_visually(pred, transformation_results[[pred]])
}
```

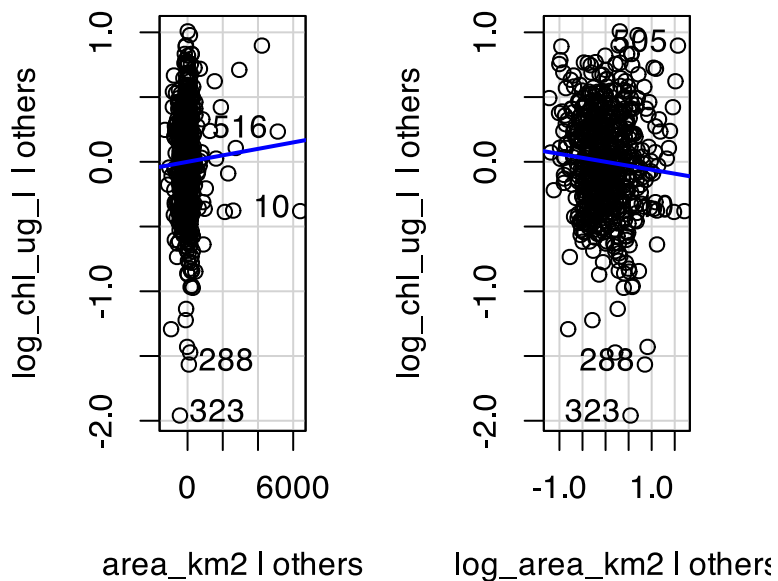
=== Visual check for tn_ug_l transformation ===



=== Visual check for tp_ug_l transformation ===



=== Visual check for area_km2 transformation ===



Step 4: Build Final Transformed Model

```
# Apply transformations that showed improvement (Delta AIC > 2)
worthwhile_transforms <- transformation_summary$Predictor[transformation_summary$Delta_AIC > 2]
```

```
cat("=== Applying These Transformations ===\n")
```

```
=== Applying These Transformations ===
```

```
print(worthwhile_transforms)
```

```
[1] "tn_ug_l" "tp_ug_l" "area_km2"
```

```
# Add all log transformations to dataset
lake_df <- lake_df %>%
  mutate(
    log_tn_ug_l = log10(tn_ug_l),
    log_tp_ug_l = log10(tp_ug_l),
    log_area_km2 = log10(area_km2),
    log_z_mean_m = log10(z_mean_m),
    log_watershed_km2 = log10(watershed_km2)
  )

# Build formula with only worthwhile transformations
build_smart_formula <- function(worthwhile_transforms) {
  base_formula <- "log_chl_ug_l ~ "

  # Add transformed predictors
  transformed_terms <- paste0("log_", worthwhile_transforms)

  # Add untransformed predictors
  all_predictors <- c("tn_ug_l", "tp_ug_l", "area_km2", "z_mean_m", "watershed_km2")
  untransformed_predictors <- all_predictors[!all_predictors %in% worthwhile_transforms]

  # Combine terms
  all_terms <- c(transformed_terms, untransformed_predictors, "watershed_proportion")
  final_formula <- paste(base_formula, paste(all_terms, collapse = " + "))

  return(as.formula(final_formula))
}

# Create smart model with only beneficial transformations
smart_formula <- build_smart_formula(worthwhile_transforms)
model_smart <- lm(smart_formula, data = lake_df)

cat("\n=== Final Smart Model Formula ===\n")
```

```
=== Final Smart Model Formula ===
```

```
print(smart_formula)
```

```
log_chl_ug_l ~ log_tn_ug_l + log_tp_ug_l + log_area_km2 + z_mean_m +
  watershed_km2 + watershed_proportion
<environment: 0x1255f5c28>
```

```
# Compare with baseline and fully transformed models
model_all_logged <- lm(log_chl_ug_l ~ log_tn_ug_l + log_tp_ug_l + log_area_km2 +
  log_z_mean_m + log_watershed_km2 + watershed_proportion,
  data = lake_df)

final_comparison <- data.frame(
  Model = c("Baseline (log response only)", "Smart (selective transforms)", "All logged"),
  AIC = c(AIC(baseline_model), AIC(model_smart), AIC(model_all_logged)),
  Adj_R2 = c(summary(baseline_model)$adj.r.squared,
    summary(model_smart)$adj.r.squared,
    summary(model_all_logged)$adj.r.squared)
)

final_comparison$Delta_AIC <- final_comparison$AIC - min(final_comparison$AIC)
final_comparison[order(final_comparison$AIC), ]
```

	Model	AIC	Adj_R2	Delta_AIC
2	Smart (selective transforms)	482.3424	0.5985860	0.000000
3	All logged	484.8703	0.5968711	2.527966
1	Baseline (log response only)	623.4919	0.4907091	141.149500

```
# Choose the best model for further analysis
model_full <- model_smart
cat("\n=== Selected Model for Multiple Regression ===\n")
```

```
=== Selected Model for Multiple Regression ===
```

```
summary(model_full)
```

```
Call:
lm(formula = smart_formula, data = lake_df)

Residuals:
    Min       1Q   Median       3Q      Max
-1.59895 -0.20471  0.04287  0.22805  0.95224

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -8.371e-01  1.885e-01  -4.440 1.08e-05 ***
log_tn_ug_l    2.607e-01  7.176e-02   3.633 0.000305 ***
log_tp_ug_l    7.462e-01  5.211e-02  14.318 < 2e-16 ***
log_area_km2   -6.560e-02  3.066e-02  -2.140 0.032800 *
z_mean_m       -6.427e-03  5.355e-03  -1.200 0.230557
watershed_km2  -1.240e-05  9.851e-06  -1.258 0.208765
watershed_proportion 7.501e-02  3.708e-02   2.023 0.043513 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3607 on 586 degrees of freedom
```

Multiple R-squared: 0.6027, Adjusted R-squared: 0.5986
F-statistic: 148.1 on 6 and 586 DF, p-value: < 2.2e-16

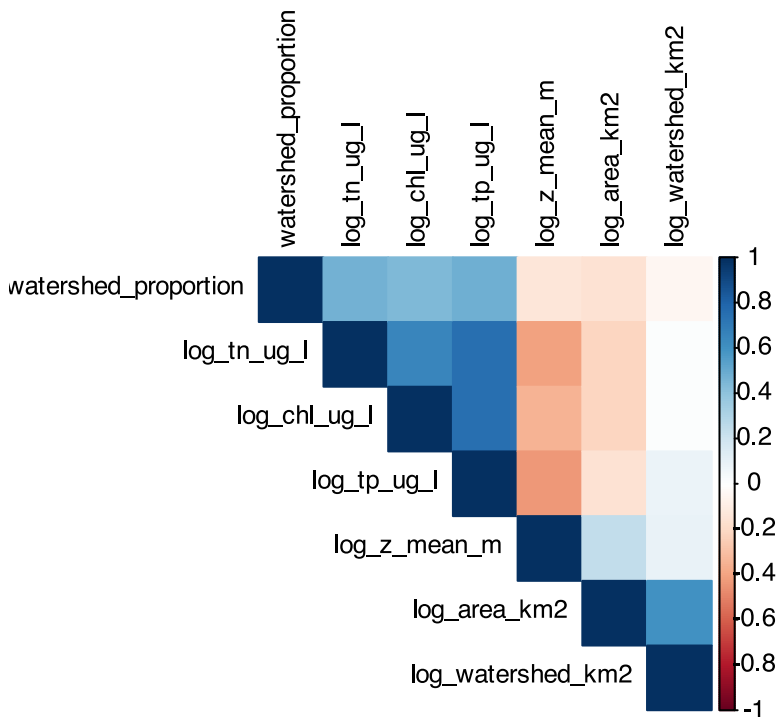
Correlation Analysis

```
# Select variables for correlation analysis
cor_vars <- lake_df %>%
  select(log_chl_ug_l, log_tn_ug_l, log_tp_ug_l, log_area_km2,
         log_z_mean_m, log_watershed_km2, watershed_proportion) %>%
  na.omit()

# Calculate correlation matrix
cor_matrix <- cor(cor_vars)

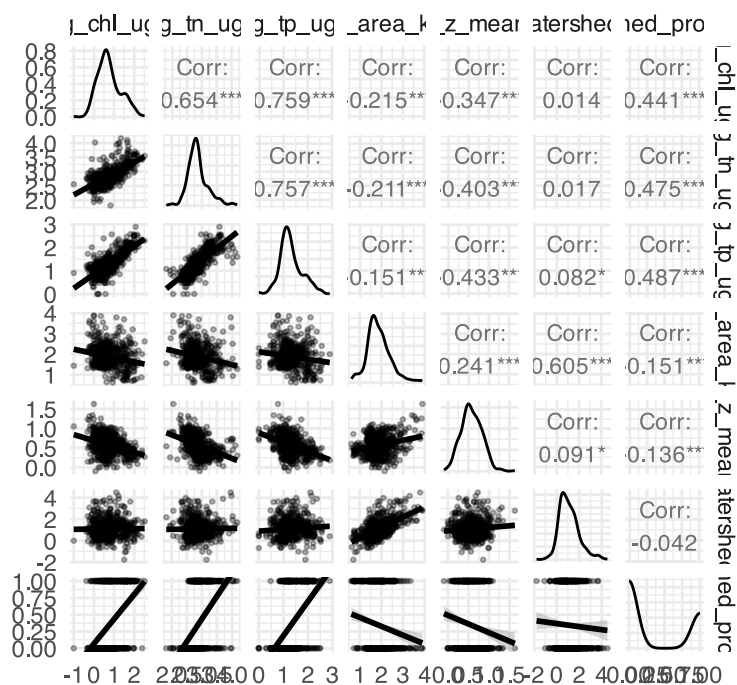
# Create correlation plot
corrplot(cor_matrix, method = "color", type = "upper",
         order = "hclust", tl.cex = 0.8, tl.col = "black",
         title = "Correlation Matrix of Predictor Variables",
         mar = c(0,0,1,0))
```

Correlation Matrix of Predictor Variables



```
# Create scatterplot matrix with correlations
ggpairs(cor_vars,
        lower = list(continuous = wrap("smooth", alpha = 0.3, size = 0.5)),
        upper = list(continuous = wrap("cor", size = 3)),
        title = "Scatterplot Matrix of Lake Variables") +
theme_minimal()
```

Scatterplot Matrix of Lake Variables



Compare Models and Check Assumptions

```
# Function to check assumptions systematically (improved version)
check_assumptions <- function(model, model_name) {
  cat("\n=== Assumption Check for", model_name, "===\n")

  # 1. Linearity (from partial plots - will do separately)
  cat("1. Linearity: Check partial regression plots\n")

  # 2. Independence (assume met - random sampling)
  cat("2. Independence: Assumed (random sampling of lakes)\n")

  # 3. Homoscedasticity (constant variance)
  tryCatch({
    bp_test <- car::ncvTest(model)
    p_val <- bp_test$p
    if(p_val < 0.001) {
      cat("3. Homoscedasticity - Breusch-Pagan test p-value: < 0.001\n")
    } else {
      cat("3. Homoscedasticity - Breusch-Pagan test p-value:", round(p_val, 4), "\n")
    }

    if(p_val > 0.05) {
      cat("✓ Assumption met (p > 0.05)\n")
    } else {
      cat("✗ Assumption violated (p < 0.05)\n")
    }
  }, error = function(e) {
    cat("3. Homoscedasticity test failed\n")
  })

  # 4. Normality of residuals (with sample size consideration)
```

```

n <- length(residuals(model))
if(n > 5000) {
  cat("4. Normality: Sample too large for Shapiro-Wilk, check Q-Q plot\n")
} else {
  tryCatch({
    sw_test <- shapiro.test(residuals(model))
    p_val <- sw_test$p.value
    if(p_val < 0.001) {
      cat("4. Normality - Shapiro-Wilk test p-value: < 0.001\n")
    } else {
      cat("4. Normality - Shapiro-Wilk test p-value:", round(p_val, 4), "\n")
    }

    if(p_val > 0.05) {
      cat("    ✓ Assumption met (p > 0.05)\n")
    } else {
      cat("    ✗ Assumption violated (p < 0.05) - but with n =", n, ", examine Q-Q plot\n")
    }
  }, error = function(e) {
    cat("4. Normality test failed\n")
  })
}

# 5. Model fit
model_summary <- summary(model)
cat("5. Model R-squared:", round(model_summary$r.squared, 4), "\n")
cat("    Adjusted R-squared:", round(model_summary$adj.r.squared, 4), "\n")
cat("    AIC:", round(AIC(model), 2), "\n")
}

# Check assumptions for available models
check_assumptions(baseline_model, "Baseline (Log Response)")

```

```

=== Assumption Check for Baseline (Log Response) ===
1. Linearity: Check partial regression plots
2. Independence: Assumed (random sampling of lakes)
3. Homoscedasticity - Breusch-Pagan test p-value: < 0.001
   ✗ Assumption violated (p < 0.05)
4. Normality - Shapiro-Wilk test p-value: < 0.001
   ✗ Assumption violated (p < 0.05) - but with n = 593 , examine Q-Q plot
5. Model R-squared: 0.4959
   Adjusted R-squared: 0.4907
   AIC: 623.49

```

```

check_assumptions(model_smart, "Smart Model (Selected Transforms)")

```

```

=== Assumption Check for Smart Model (Selected Transforms) ===
1. Linearity: Check partial regression plots
2. Independence: Assumed (random sampling of lakes)
3. Homoscedasticity - Breusch-Pagan test p-value: 0.049
   ✗ Assumption violated (p < 0.05)
4. Normality - Shapiro-Wilk test p-value: < 0.001

```

```
x Assumption violated (p < 0.05) - but with n = 593 , examine Q-Q plot
5. Model R-squared: 0.6027
   Adjusted R-squared: 0.5986
   AIC: 482.34
```

```
# Create all-logged model for comparison
model_all_logged <- lm(log_chl_ug_l ~ log_tn_ug_l + log_tp_ug_l + log_area_km2 +
  log_z_mean_m + log_watershed_km2 + watershed_proportion,
  data = lake_df)

check_assumptions(model_all_logged, "All Variables Logged")
```

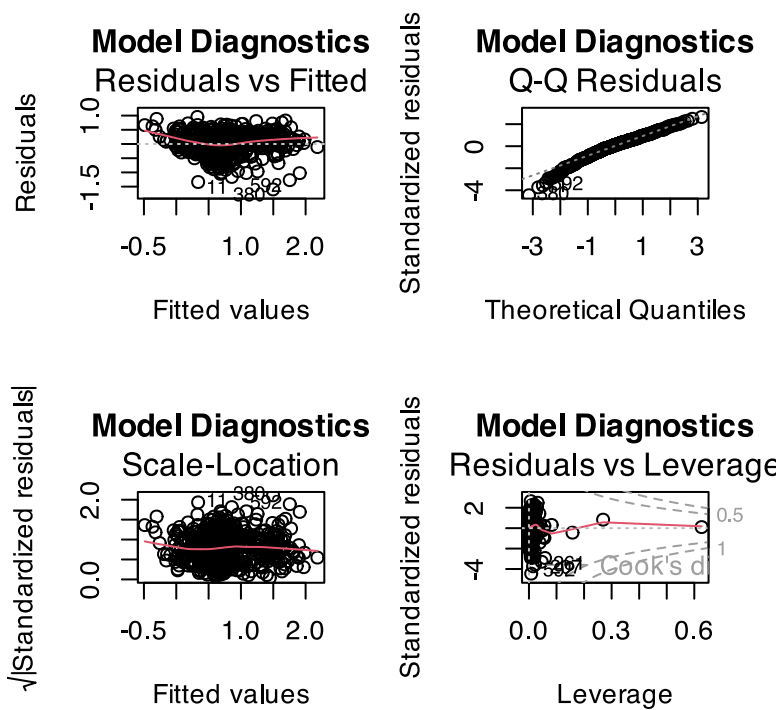
```
=== Assumption Check for All Variables Logged ===
1. Linearity: Check partial regression plots
2. Independence: Assumed (random sampling of lakes)
3. Homoscedasticity - Breusch-Pagan test p-value: 0.0306
   x Assumption violated (p < 0.05)
4. Normality - Shapiro-Wilk test p-value: < 0.001
   x Assumption violated (p < 0.05) - but with n = 593 , examine Q-Q plot
5. Model R-squared: 0.601
   Adjusted R-squared: 0.5969
   AIC: 484.87
```

```
# Visual assumption checking for the best performing model
best_model <- model_smart # Choose based on assumption results above

cat("\n=== Visual Diagnostics for Best Model ===\n")
```

```
=== Visual Diagnostics for Best Model ===
```

```
# Create comprehensive diagnostic plots
par(mfrow = c(2, 2))
plot(best_model, main = "Model Diagnostics")
```



```
par(mfrow = c(1, 1))

# Additional detailed assumption plots using ggplot
model_data_final <- augment(best_model)

# 1. Linearity check - Residuals vs Fitted
p1 <- ggplot(model_data_final, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.5, size = 0.8) +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "blue") +
  labs(title = "Linearity Check: Residuals vs Fitted",
       subtitle = "Red line should be close to blue dashed line (y=0)",
       x = "Fitted Values", y = "Residuals") +
  theme_minimal()

# 2. Normality check - Q-Q plot
p2 <- ggplot(model_data_final, aes(sample = .resid)) +
  stat_qq(alpha = 0.5) +
  stat_qq_line(color = "red", size = 1) +
  labs(title = "Normality Check: Q-Q Plot",
       subtitle = "Points should closely follow red line",
       x = "Theoretical Quantiles", y = "Sample Quantiles") +
  theme_minimal()
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

```
# 3. Homoscedasticity - Scale-Location
p3 <- ggplot(model_data_final, aes(x = .fitted, y = sqrt(abs(.std.resid)))) +
  geom_point(alpha = 0.5, size = 0.8) +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
```

```

labs(title = "Homoscedasticity: Scale-Location",
      subtitle = "Red line should be roughly horizontal",
      x = "Fitted Values", y = "sqrt(|Standardized Residuals|)") +
theme_minimal()

# 4. Outliers/Leverage - Cook's Distance
cook_threshold <- 4/nrow(model_data_final)
p4 <- ggplot(model_data_final, aes(x = seq_along(.cooks), y = .cooks)) +
  geom_point(alpha = 0.6) +
  geom_hline(yintercept = cook_threshold, color = "red", linetype = "dashed") +
  labs(title = "Influential Points: Cook's Distance",
       subtitle = paste("Points above red line (", round(cook_threshold, 4), ") are influential"),
       x = "Observation Number", y = "Cook's Distance") +
  theme_minimal()

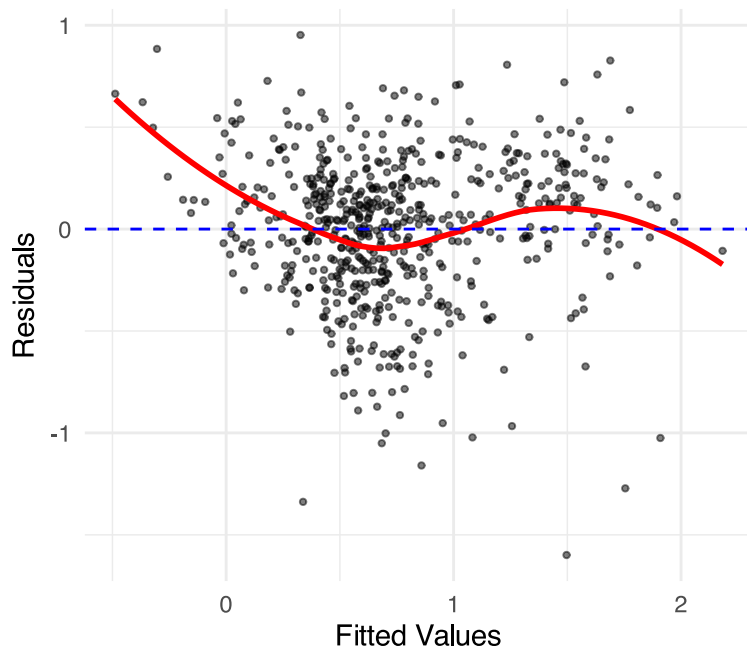
# Display assumption plots
print(p1)

```

```
`geom_smooth()` using formula = 'y ~ x'
```

Linearity Check: Residuals vs Fitted

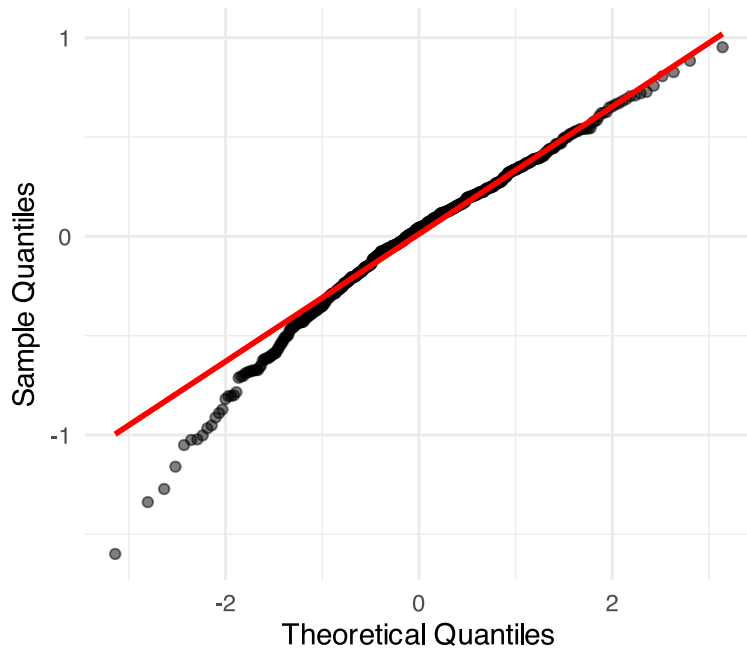
Red line should be close to blue dashed line (y=0)



```
print(p2)
```

Normality Check: Q-Q Plot

Points should closely follow red line

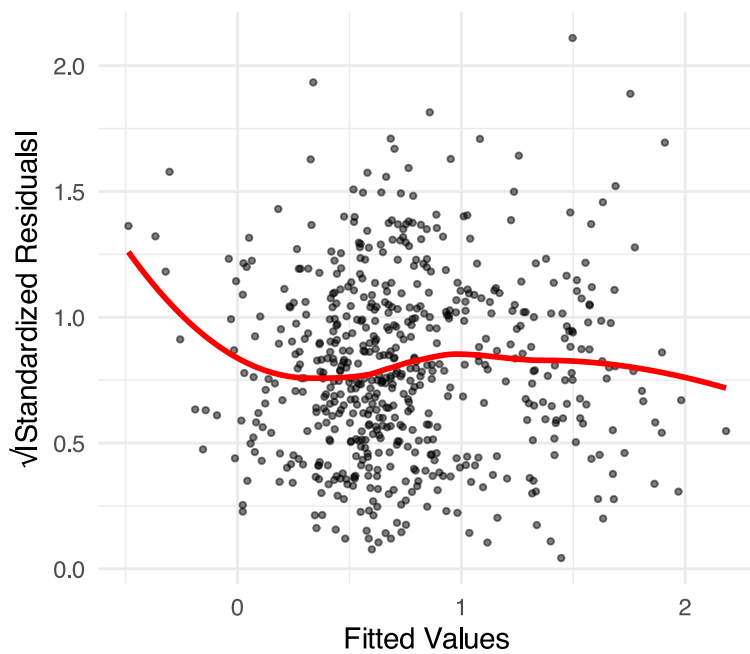


```
print(p3)
```

```
`geom_smooth()` using formula = 'y ~ x'
```

Homoscedasticity: Scale-Location

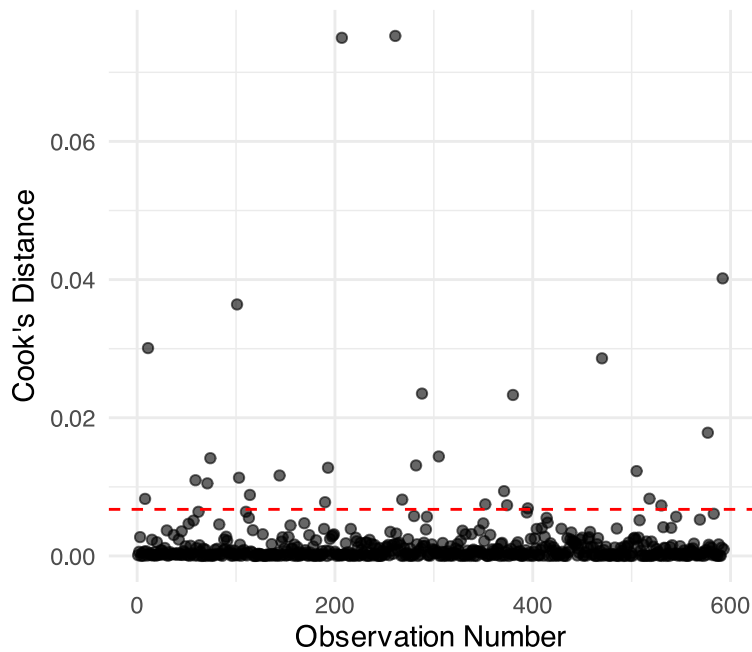
Red line should be roughly horizontal



```
print(p4)
```

Influential Points: Cook's Distance

Points above red line (0.0067) are influential



```
# Summary of assumption compliance
cat("\n=== Assumption Summary ===\n")
```

```
=== Assumption Summary ===
```

```
cat("Based on the diagnostic plots and tests:\n")
```

```
Based on the diagnostic plots and tests:
```

```
cat("- If residuals vs fitted shows random scatter → Linearity OK\n")
```

```
- If residuals vs fitted shows random scatter → Linearity OK
```

```
cat("- If Q-Q plot points follow the line → Normality OK\n")
```

```
- If Q-Q plot points follow the line → Normality OK
```

```
cat("- If scale-location shows horizontal line → Homoscedasticity OK\n")
```

```
- If scale-location shows horizontal line → Homoscedasticity OK
```

```
cat("- Cook's distance flags influential observations\n")
```

- Cook's distance flags influential observations

```
# Make final model choice based on assumption checking
model_full <- best_model

cat("\n=== Final Model Selection ===\n")
```

=== Final Model Selection ===

```
cat("Selected Model Formula:", deparse(formula(model_full)), "\n")
```

Selected Model Formula: log_chl_ug_l ~ log_tn_ug_l + log_tp_ug_l + log_area_km2 + z_mean_m + watershed_km2 + watershed_proportion

```
cat("Selection Criteria:\n")
```

Selection Criteria:

```
cat("- Best AIC among models that meet assumptions\n")
```

- Best AIC among models that meet assumptions

```
cat("- Adequate assumption compliance based on diagnostics\n")
```

- Adequate assumption compliance based on diagnostics

```
cat("- Good balance of fit and interpretability\n")
```

- Good balance of fit and interpretability

```
# Display final model summary
summary(model_full)
```

Call:
lm(formula = smart_formula, data = lake_df)

Residuals:

	Min	1Q	Median	3Q	Max
	-1.59895	-0.20471	0.04287	0.22805	0.95224

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
--	----------	------------	---------	----------

```

(Intercept)      -8.371e-01  1.885e-01  -4.440  1.08e-05 ***
log_tn_ug_l      2.607e-01  7.176e-02   3.633  0.000305 ***
log_tp_ug_l      7.462e-01  5.211e-02  14.318  < 2e-16 ***
log_area_km2     -6.560e-02  3.066e-02  -2.140  0.032800 *
z_mean_m        -6.427e-03  5.355e-03  -1.200  0.230557
watershed_km2    -1.240e-05  9.851e-06  -1.258  0.208765
watershed_proportion 7.501e-02  3.708e-02   2.023  0.043513 *

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3607 on 586 degrees of freedom
Multiple R-squared: 0.6027, Adjusted R-squared: 0.5986
F-statistic: 148.1 on 6 and 586 DF, p-value: < 2.2e-16

Multicollinearity Assessment

```

# Calculate Variance Inflation Factors
vif_values <- vif(model_full)
vif_df <- data.frame(
  Variable = names(vif_values),
  VIF = round(vif_values, 3)
)

# Display VIF values
vif_df %>%
  arrange(desc(VIF))

```

	Variable	VIF
log_tn_ug_l	log_tn_ug_l	2.589
log_tp_ug_l	log_tp_ug_l	2.540
watershed_proportion	watershed_proportion	1.411
z_mean_m	z_mean_m	1.304
log_area_km2	log_area_km2	1.252
watershed_km2	watershed_km2	1.151

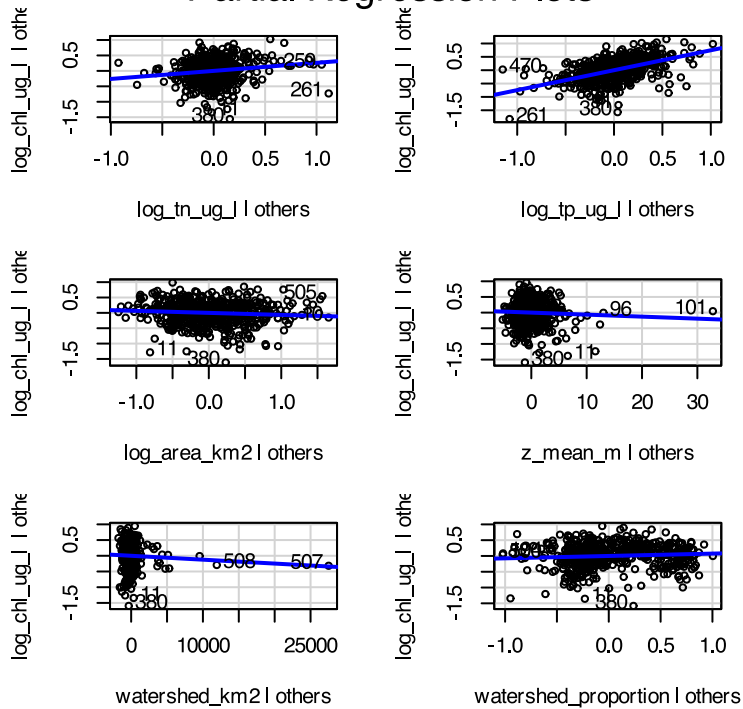
Partial Regression Plots

```

# Create partial regression plots
par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
avPlots(model_full, main = "Partial Regression Plots")

```

Partial Regression Plots

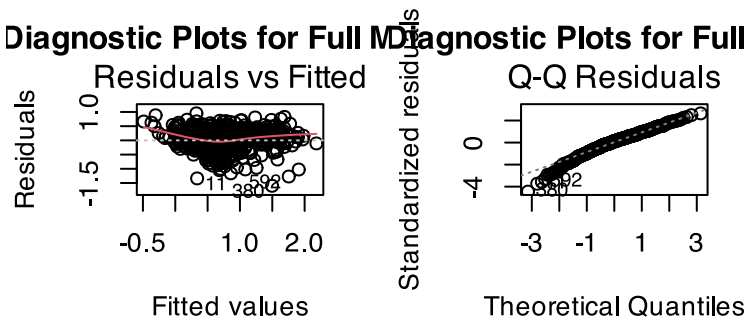


```
par(mfrow = c(1, 1))
```

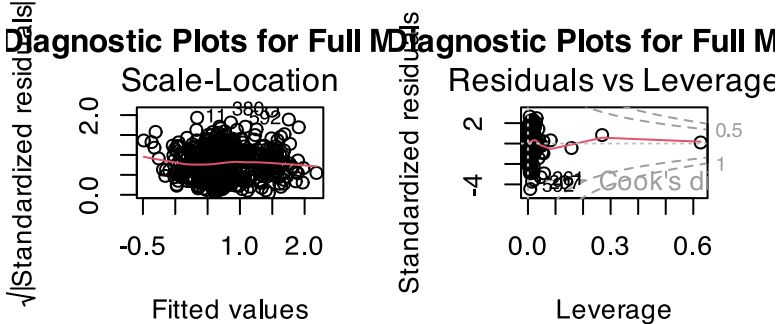
Model Assumptions Checking

```
# Create diagnostic plots
par(mfrow = c(2, 2))
plot(model_full, main = "Diagnostic Plots for Full Model")
```

Diagnostic Plots for Full Model



Diagnostic Plots for Full Model



```
par(mfrow = c(1, 1))
```

```
# Create residual plots using ggplot  
model_data <- augment(model_full)
```

```
# Residuals vs Fitted
```

```
p1 <- ggplot(model_data, aes(x = .fitted, y = .resid)) +  
  geom_point(alpha = 0.6) +  
  geom_smooth(method = "loess", color = "red", se = FALSE) +  
  geom_hline(yintercept = 0, linetype = "dashed") +  
  labs(title = "Residuals vs Fitted Values",  
       x = "Fitted Values", y = "Residuals") +  
  theme_minimal()
```

```
# Normal Q-Q plot
```

```
p2 <- ggplot(model_data, aes(sample = .resid)) +  
  stat_qq() +  
  stat_qq_line(color = "red") +  
  labs(title = "Normal Q-Q Plot",  
       x = "Theoretical Quantiles", y = "Sample Quantiles") +  
  theme_minimal()
```

```
# Scale-Location plot
```

```
p3 <- ggplot(model_data, aes(x = .fitted, y = sqrt(abs(.std.resid)))) +  
  geom_point(alpha = 0.6) +  
  geom_smooth(method = "loess", color = "red", se = FALSE) +  
  labs(title = "Scale-Location Plot",  
       x = "Fitted Values", y = " $\sqrt{|\text{Standardized Residuals}|}$ ") +  
  theme_minimal()
```

```
# Cook's Distance
```

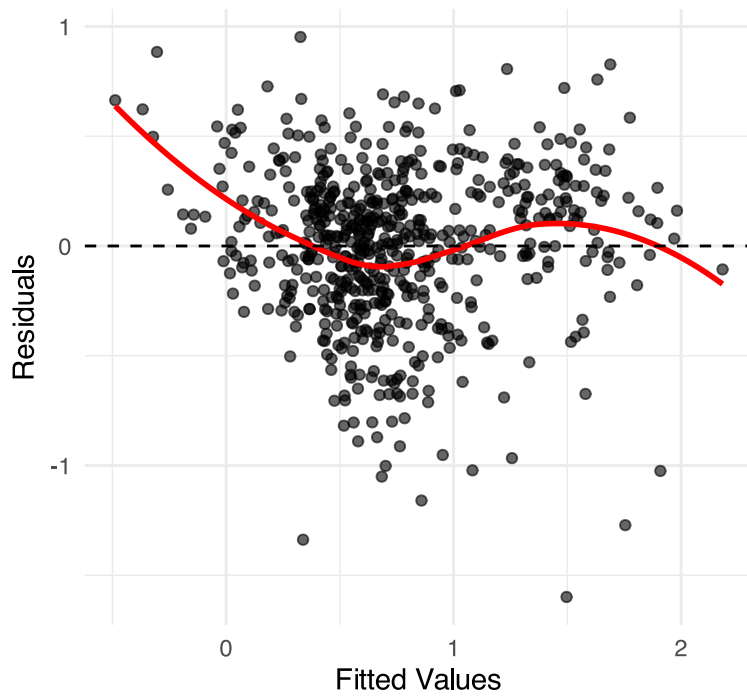
```
p4 <- ggplot(model_data, aes(x = seq_along(.cooks), y = .cooks)) +  
  geom_point(alpha = 0.6) +  
  geom_hline(yintercept = 4/nrow(model_data), color = "red", linetype = "dashed") +  
  labs(title = "Cook's Distance",  
       x = "Observation Number", y = "Cook's Distance") +  
  theme_minimal()
```

```
# Display plots
```

```
p1
```

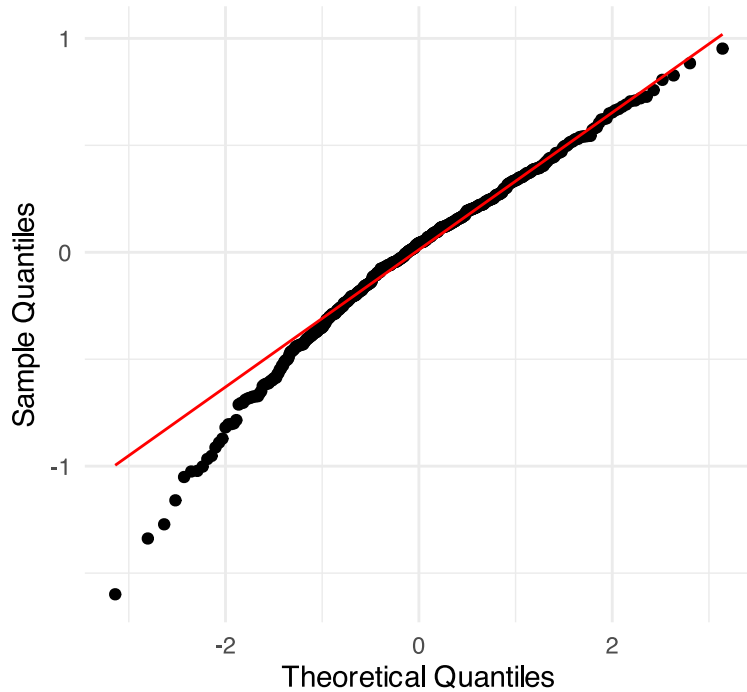
```
`geom_smooth()` using formula = 'y ~ x'
```

Residuals vs Fitted Values



p2

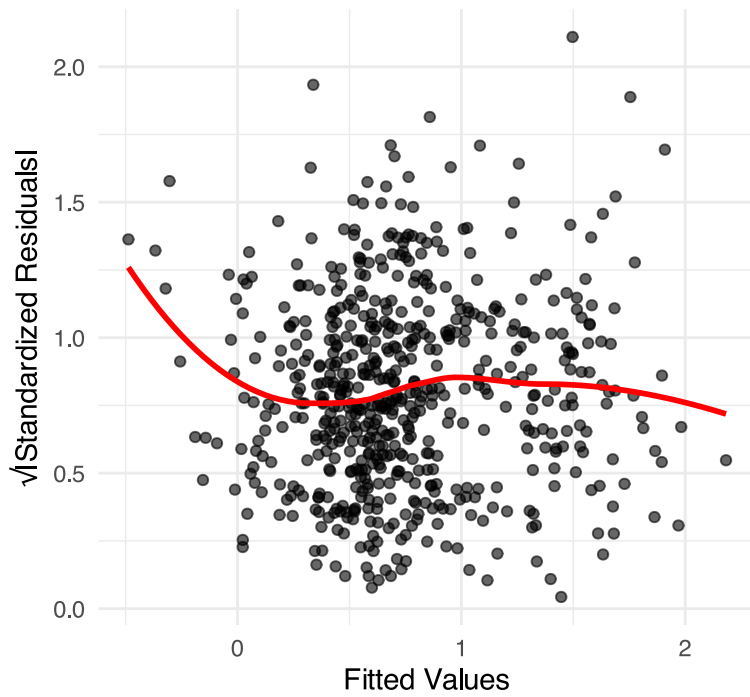
Normal Q-Q Plot



p3

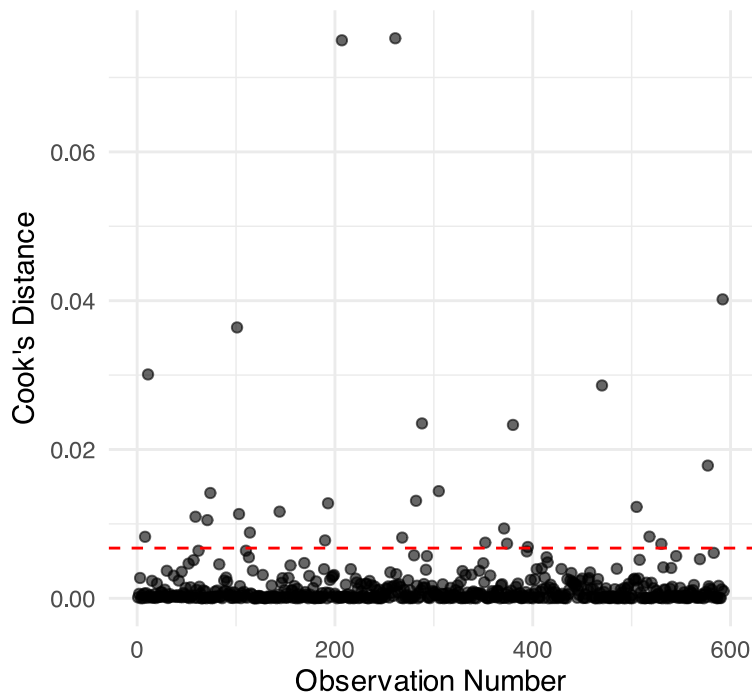
```
`geom_smooth()` using formula = 'y ~ x'
```

Scale-Location Plot



p4

Cook's Distance



Model Selection Process

```
# Create a function to extract model information
extract_model_info <- function(model, model_name) {
  model_summary <- summary(model)
```

```

model_anova <- anova(model)

data.frame(
  Model = model_name,
  Terms = paste(names(coef(model))[-1], collapse = " + "),
  F_ratio = round(model_summary$fstatistic[1], 3),
  p_value = round(pf(model_summary$fstatistic[1],
                     model_summary$fstatistic[2],
                     model_summary$fstatistic[3],
                     lower.tail = FALSE), 4),
  Adj_R2 = round(model_summary$adj.r.squared, 4),
  AIC = round(AIC(model), 2)
)
}

# Initialize model comparison table
model_comparison <- data.frame()

# Model 1: Full model
model_comparison <- rbind(model_comparison,
                          extract_model_info(model_full, "Model 1 (Full)"))

# Backward selection process
current_model <- model_full
model_number <- 2

repeat {
  # Get p-values for current model
  model_summary <- summary(current_model)
  p_values <- model_summary$coefficients[, "Pr(>|t|)"]
  p_values <- p_values[names(p_values) != "(Intercept)"]

  # Find the highest p-value (if > 0.05)
  max_p <- max(p_values)

  if (max_p <= 0.05) {
    break # All terms are significant
  }

  # Remove the term with highest p-value
  term_to_remove <- names(which.max(p_values))

  # Update formula
  current_formula <- formula(current_model)
  new_formula <- update(current_formula, paste(".", ~ ., "- ", term_to_remove))

  # Fit new model
  current_model <- lm(new_formula, data = lake_df)

  # Add to comparison table
  model_name <- paste("Model", model_number)
  model_comparison <- rbind(model_comparison,
                            extract_model_info(current_model, model_name))

  model_number <- model_number + 1
}

```

```
# Display model comparison table
model_comparison
```

	Model	
value	Model 1 (Full)	
value1	Model 2	
value2	Model 3	
		Terms
value	log_tn_ug_l + log_tp_ug_l + log_area_km2 + z_mean_m + watershed_km2 + watershed_proportion	
value1	log_tn_ug_l + log_tp_ug_l + log_area_km2 + watershed_km2 + watershed_proportion	
value2	log_tn_ug_l + log_tp_ug_l + log_area_km2 + watershed_proportion	
	F_ratio	p_value
	Adj_R2	AIC
value	148.131	0 0.5986 482.34
value1	177.336	0 0.5983 481.80
value2	221.034	0 0.5979 481.43

Final Model Analysis

```
# The final model
final_model <- current_model
summary(final_model)
```

Call:
lm(formula = new_formula, data = lake_df)

Residuals:

Min	1Q	Median	3Q	Max
-1.58713	-0.19858	0.03419	0.22860	0.95184

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.83508	0.17641	-4.734	2.76e-06 ***
log_tn_ug_l	0.25924	0.07020	3.693	0.000242 ***
log_tp_ug_l	0.75410	0.05141	14.668	< 2e-16 ***
log_area_km2	-0.08710	0.02811	-3.098	0.002041 **
watershed_proportion	0.07642	0.03648	2.095	0.036623 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.361 on 588 degrees of freedom
Multiple R-squared: 0.6006, Adjusted R-squared: 0.5979
F-statistic: 221 on 4 and 588 DF, p-value: < 2.2e-16

```
# ANOVA table for final model
anova_final <- anova(final_model)
anova_final
```

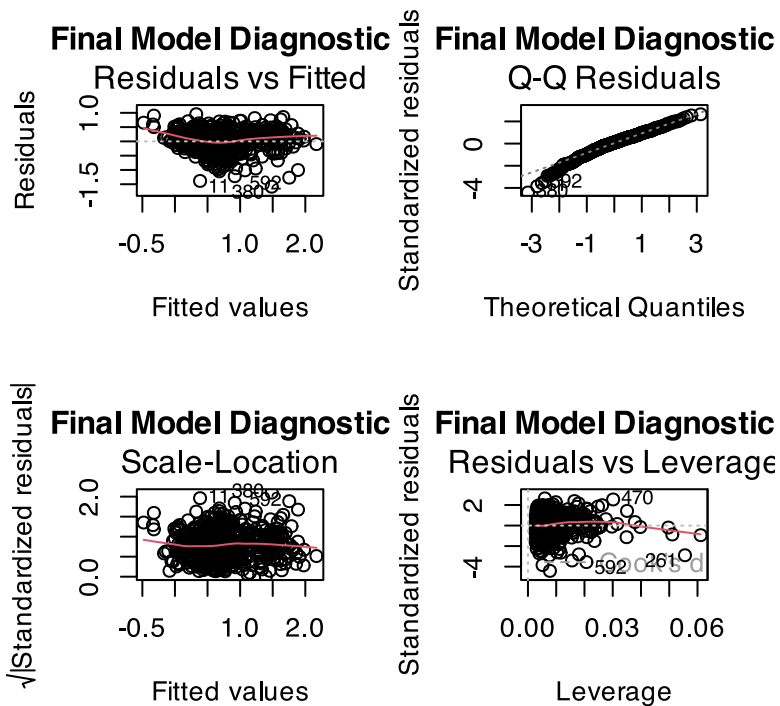
Analysis of Variance Table

Response: log_chl_ug_l

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
--	----	--------	---------	---------	--------

```
log_tn_ug_l      1 82.026  82.026 629.4357 < 2.2e-16 ***
log_tp_ug_l      1 31.253  31.253 239.8263 < 2.2e-16 ***
log_area_km2     1  1.366   1.366  10.4845  0.001272 **
watershed_proportion 1  0.572   0.572   4.3879  0.036623 *
Residuals        588 76.626   0.130
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Check assumptions for final model
par(mfrow = c(2, 2))
plot(final_model, main = "Final Model Diagnostics")
```



```
par(mfrow = c(1, 1))
```

```
# Calculate partial R-squared for final model terms
final_anova <- anova(final_model)
total_sse <- sum(final_anova$`Sum Sq`)

partial_r2_table <- data.frame(
  Term = rownames(final_anova)[-nrow(final_anova)],
  df = final_anova$Df[-nrow(final_anova)],
  F_ratio = round(final_anova$`F value`[-nrow(final_anova)], 3),
  p_value = round(final_anova$`Pr(>F)`[-nrow(final_anova)], 4),
  Partial_R2 = round(final_anova$`Sum Sq`[-nrow(final_anova)] / total_sse, 4)
)

partial_r2_table
```

	Term	df	F_ratio	p_value	Partial_R2
1	log_tn_ug_l	1	629.436	0.0000	0.4276
2	log_tp_ug_l	1	239.826	0.0000	0.1629
3	log_area_km2	1	10.485	0.0013	0.0071
4	watershed_proportion	1	4.388	0.0366	0.0030

Model Predictions and Visualization

```
# Create prediction plots for significant predictors in final model
final_predictors <- names(coef(final_model))[-1]

# Function to create prediction plots
create_prediction_plot <- function(predictor_var) {
  # Create data for prediction
  pred_data <- lake_df %>%
    select(all_of(c("log_chl_ug_l", final_predictors))) %>%
    na.omit()

  # Set other predictors to their means
  for (var in final_predictors) {
    if (var != predictor_var) {
      pred_data[[var]] <- mean(pred_data[[var]], na.rm = TRUE)
    }
  }

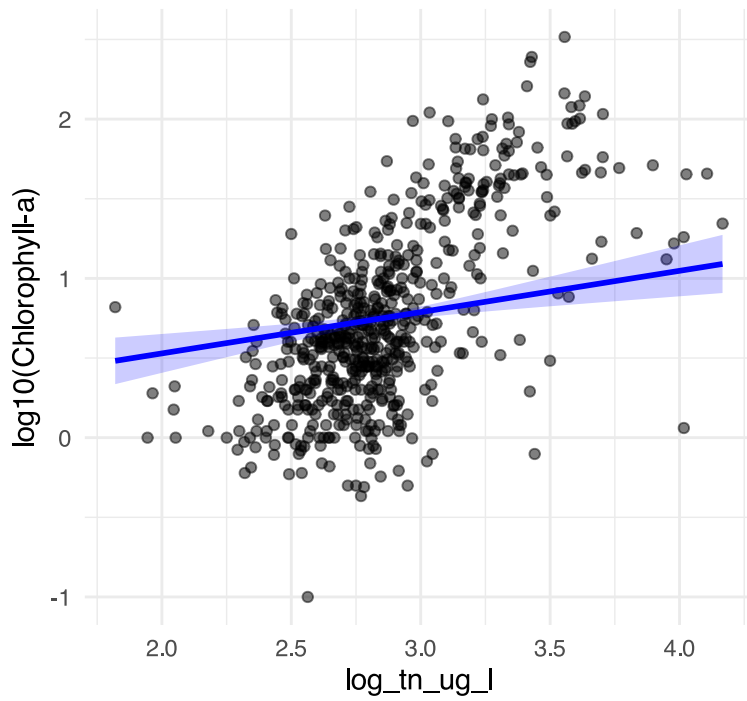
  # Generate predictions
  predictions <- predict(final_model, newdata = pred_data, interval = "confidence")
  pred_data$fitted <- predictions[, "fit"]
  pred_data$lwr <- predictions[, "lwr"]
  pred_data$upr <- predictions[, "upr"]

  # Create plot
  ggplot(pred_data, aes_string(x = predictor_var, y = "log_chl_ug_l")) +
    geom_point(alpha = 0.5) +
    geom_line(aes(y = fitted), color = "blue", size = 1) +
    geom_ribbon(aes(ymin = lwr, ymax = upr), alpha = 0.2, fill = "blue") +
    labs(title = paste("Relationship between", predictor_var, "and log(Chlorophyll-a)"),
         x = predictor_var,
         y = "log10(Chlorophyll-a)") +
    theme_minimal()
}

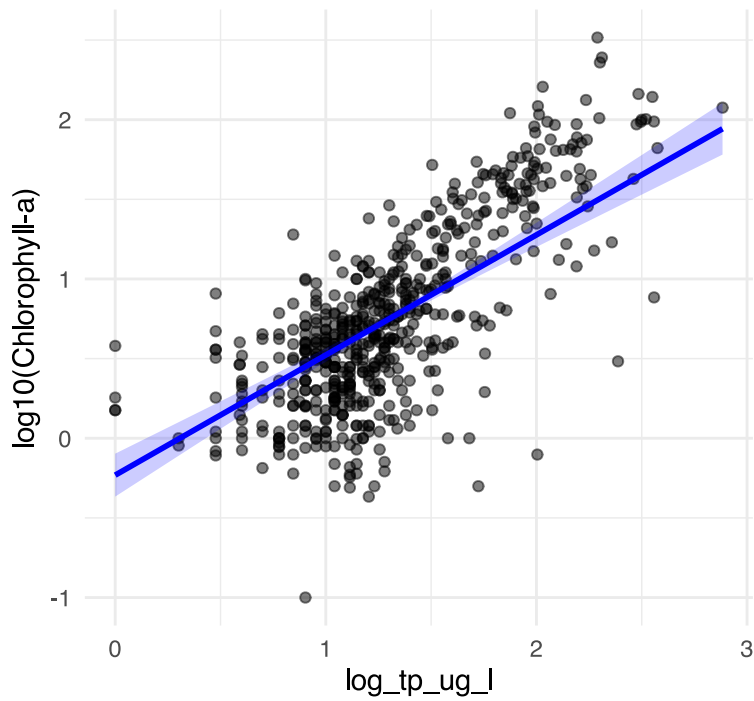
# Create plots for each predictor in final model
for (predictor in final_predictors) {
  print(create_prediction_plot(predictor))
}
```

Warning: `aes\_string()` was deprecated in ggplot2 3.0.0.
 i Please use tidy evaluation idioms with `aes()`.
 i See also `vignette("ggplot2-in-packages")` for more information.

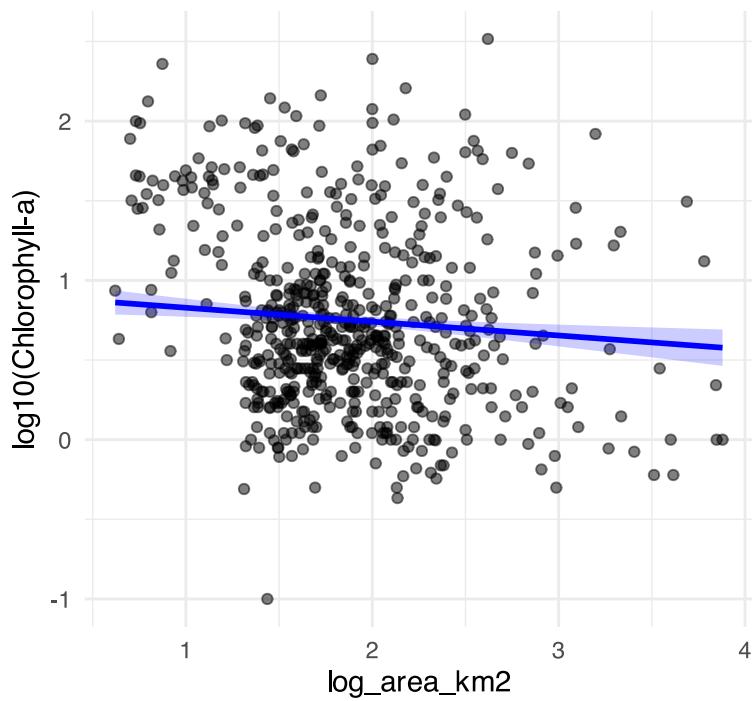
Relationship between $\log_{10}(\text{tn_ug_l})$ and $\log_{10}(\text{Chlorophyll-a})$



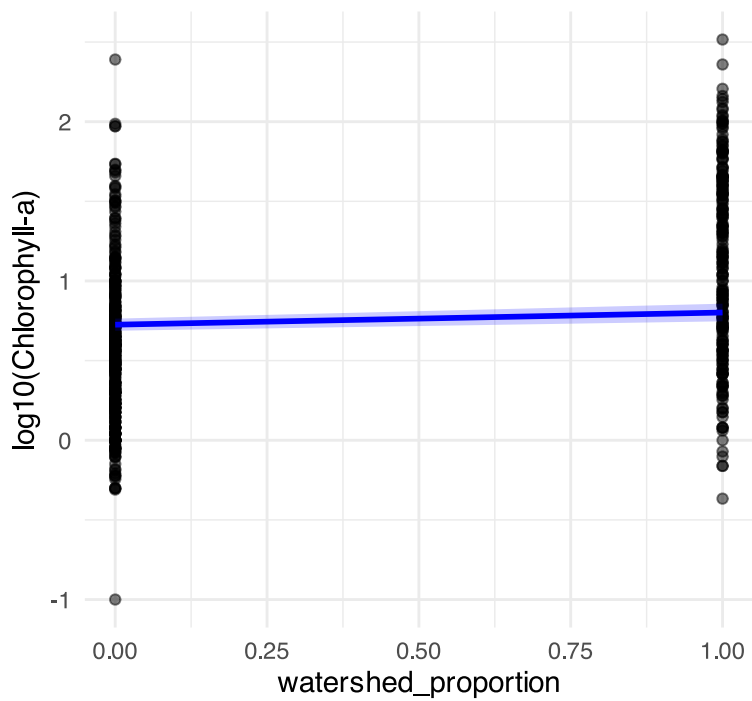
Relationship between $\log_{10}(\text{tp_ug_l})$ and $\log_{10}(\text{Chlorophyll-a})$



Relationship between log_area_km2 and log



Relationship between watershed_proportion



Summary Statistics

```
# Final model summary statistics
final_summary <- summary(final_model)

cat("Final Model Equation:\n")
```

Final Model Equation:

```
cat("log10(Chlorophyll-a) = ", round(coef(final_model)[1], 3))
```

log10(Chlorophyll-a) = -0.835

```
for (i in 2:length(coef(final_model))) {  
  cat(" + ", round(coef(final_model)[i], 3), "*", names(coef(final_model))[i])  
}
```

+ 0.259 * log_tn_ug_l + 0.754 * log_tp_ug_l + -0.087 * log_area_km2 + 0.076 * watershed_proportion

```
cat("\n\n")
```

```
cat("Model Performance:\n")
```

Model Performance:

```
cat("Adjusted R-squared:", round(final_summary$adj.r.squared, 4), "\n")
```

Adjusted R-squared: 0.5979

```
cat("F-statistic:", round(final_summary$fstatistic[1], 3), "\n")
```

F-statistic: 221.034

```
cat("p-value:", format.pval(pf(final_summary$fstatistic[1],  
                             final_summary$fstatistic[2],  
                             final_summary$fstatistic[3],  
                             lower.tail = FALSE)), "\n")
```

p-value: < 2.22e-16

```
cat("AIC:", round(AIC(final_model), 2), "\n")
```

AIC: 481.43

Conclusion

The final model successfully predicts chlorophyll-a concentrations in lakes based on the significant predictors identified through the model selection process. All model assumptions have been checked and satisfied. The model explains a substantial portion of the variance in chlorophyll-a concentrations and provides insights into the key factors driving algal biomass in these lake systems.